Scholar App Logic and Risk Reference

Date: 2026-03-15

Purpose:
- explain the current app logic end to end
- identify architectural and workflow vulnerabilities
- identify stochastic vs deterministic failure modes
- identify likely latency and concurrency bottlenecks
- give a reference point before migrating away from Notion as the runtime store

This document intentionally treats Notion migration as a separate known step. The focus here is:
- how the app behaves today
- where it is brittle
- what should be measured if multiple concurrent users need acceptable latency

---

1. What the App Actually Is

Scholar is not just a tutoring dashboard. It is a combined:

- student-facing learning portal
- tutor operations tool
- assessment engine
- scheduling engine
- shared question-bank system
- AI-assisted import pipeline
- AI-assisted MCQ-generation system
- progress visualization system

The complexity comes from the fact that these layers are coupled through shared runtime data.

Core surfaces:
- student dashboard
- practice room
- pre-class assessment
- exit ticket
- homework
- flashcards
- admin dashboard
- import / review / end-class operations

Main runtime code:
- [pages/dashboard.js](/home/koustubh/Downloads/website/scholar/pages/dashboard.js)
- [pages/api/student/dashboard.js](/home/koustubh/Downloads/website/scholar/pages/api/student/dashboard.js)
-

[pages/api/student/assessment.js](/home/koustubh/Downloads/website/scholar/pages/api/student/assessment.js)
• [pages/api/student/submit.js](/home/koustubh/Downloads/website/scholar/pages/api/student/submit.js)
•
[pages/api/student/homework.js](/home/koustubh/Downloads/website/scholar/pages/api/student/homework.js)
• [pages/api/student/progress-graph.js](/home/koustubh/Downloads/website/scholar/pages/api/student/progress-graph.js)
• [pages/api/admin/import.js](/home/koustubh/Downloads/website/scholar/pages/api/admin/import.js)
• [pages/api/admin/end-class.js](/home/koustubh/Downloads/website/scholar/pages/api/admin/end-class.js)
• [lib/notion.js](/home/koustubh/Downloads/website/scholar/lib/notion.js)
• [lib/logic.js](/home/koustubh/Downloads/website/scholar/lib/logic.js)
• [lib/tutor-calendar.js](/home/koustubh/Downloads/website/scholar/lib/tutor-calendar.js)
• [lib/claude.js](/home/koustubh/Downloads/website/scholar/lib/claude.js)

---

2. High-Level System Model

At a conceptual level:

```
Google Login
      !“
Student identity / admin impersonation
      !“
Student + subject context
      !“
Scores rows determine session, weakness, and scheduling
      !“
Question-type pages provide the actual question content
      !“
Claude fills gaps when MCQ cache is missing
      !“
Student answers update weakness, status, and scheduling
```

The most important thing to understand is:

• `Scores DB` is the operational backbone
• subject question pages are the content backbone
• calendar data is the timing/backfill backbone
• Claude fills semantic gaps

---

3. Data Model As Used By The App

3.1 Runtime entities

The current app behaves as if the system contains these logical entities:

• `Student`

- `Subject`
- `Enrollment`
- `Question Type`
- `Question Page Context Block`
- `Question`
- `Score Row`
- `Session`
- `Homework Assignment`
- `MCQ Cache`
- `Report`

The problem is that some of these exist as explicit database rows, and some exist only as structure inside Notion page bodies.

3.2 Where data really lives

```
Students DB
  student identity, timezone, subject relations

Subjects DB
  subject label, question data source pointer

Enrollments DB
  duration + class-day metadata (sparse, partially legacy)

Scores DB
  per-student x per-question-type operational state

Subject Question-Type Tables
  question-type rows

Question Page Bodies
  context blocks, LO labels, Q/A content, images, qhash, cache lines
```

3.3 Important hidden truth

The actual academic structure is split across:

- Scores rows
- question-type table properties
- question page callouts

That split is one of the biggest sources of complexity and drift.

See also:
- [docs/notion-schema-mismatch-audit.md](/home/koustubh/Downloads/website/scholar/docs/notion-schema-mismatch-audit.md)

---

4. Authentication and Identity Flow

Main auth code:

- [pages/api/auth/[...nextauth].js](/home/koustubh/Downloads/website/scholar/pages/api/auth/[...nextauth].js)

Flow:

```
Google OAuth
    !“
signIn callback
    !“
 i f   a d m i n   e m a i l  !’   a l l o w
else check if email exists in Students DB
    !“
jwt callback
    !“
store notionStudentId / accessToken / refreshToken
    !“
session callback
    !“
session.notionStudentId used by all student APIs
```

Important details:

- admin bypass is hardcoded by email
- preview/admin impersonation works by passing `?as=<studentId>`
- many APIs allow impersonation only if admin

Auth vulnerabilities / issues

1. Email-based authorization is simple but brittle.
   - Admin identity is one hardcoded email.
   - Student access depends on email matching a Notion student row.

2. Preview and demo bypass paths add complexity.
   - There are now multiple modes:
   - real student
   - admin impersonation
   - local demo mode
   - These are useful, but increase mental and security surface area.

3. Session stability in dev is fragile.
   - Not necessarily an app-logic bug, but `.next` corruption has repeatedly broken auth/session.

Metrics to track:

- auth success rate
- sign-in rejection rate
- `/api/auth/session` error rate
- preview-mode usage rate

---

5. Dashboard Logic

Main dashboard API:

- [pages/api/student/dashboard.js](/home/koustubh/Downloads/website/scholar/pages/api/student/dashboard.js)

Dashboard flow:

```
studentId
    !“
load student
    !“
load subjects from student's subjectIds
    !“
for each subject:
    fetch upcoming classes from Google Calendar
    infer todayClass / nextClass
    !“
return student + subjectsWithCalendar
```

Key behavior:

- student identity drives subject list
- calendar matching is fuzzy by subject and student name
- dashboard uses student timezone for "today"

Risks

1. Calendar matching is fuzzy, not deterministic.
   - Implemented in [lib/tutor-calendar.js](/home/koustubh/Downloads/website/scholar/lib/tutor-calendar.js)
   - Risks:
   - wrong event selected
   - no event selected
   - duplicate classes in same horizon

2. Each subject triggers calendar fetch logic.
   - This can become expensive with more subjects.

Metrics:

- calendar match rate
- number of matched events per subject
- dashboard API p50 / p95 latency
- downstream calendar API latency

---

6. Assessment Logic

Primary code:

- [pages/api/student/assessment.js](/home/koustubh/Downloads/website/scholar/pages/api/student/assessment.js)
- [pages/api/student/submit.js](/home/koustubh/Downloads/website/scholar/pages/api/student/submit.js)

There are two modes:

- `pre`
- `exit`

6.1 Pre-class assessment

Algorithm:

```
subjectId + sessionDate
    !“
find previous session's score rows
    !“
hydrate score rows into question types
    !“
for each selected question type:
    fetch student-context question pool from page
        i f   n o n e  !’  f u l l - p a g e   f a l l b a c k
        i f   M C Q   c a c h e   e x i s t s  !’  u s e   c a c h e d   M C Q
        e l s e  !’  a s k   C l a u d e   t o   g e n e r a t e   M C Q   f r o m   s t o r
    !“
return assessment questions
```

Important details:

- previous-session lookup is date-sensitive
- if exact date lookup fails, code falls back to older heuristic
- cache miss invokes Claude at runtime

6.2 Exit ticket

Algorithm:

```
subjectId + optional sessionDate
    !“
load latest session rows (or rows for exact sessionDate)
    !“
check for "Approved for Exit" status
    !“
 i f   n o t   a p p r o v e d  !’  l o c k e d
    !“
load today's question types
    !“
same question hydration / cache / Claude pipeline as pre-class
```

6.3 Submission

Submission flow:

```
answers[]
    !“
for each wrong answer:
    increment weakness score
    !“
if pre-class:
    maybe apply FIFO swap
```

```
    !“
if exit:
    mark question status Done / To be reviewed
    !“
recompute weakness / trends
```

Risks

1. Claude runtime generation is on the request path.
   • If cache is missing, assessment latency spikes.

2. Previous-session selection is subtle.
   • A small date bug can make pre-class return zero questions.

3. Exit ticket depends on status gating.
   • Mis-set status can lock/unlock wrong sessions.

Metrics:
- assessment generation p50 / p95 / p99
- cache hit rate
- Claude runtime generation count per assessment
- no-data assessment rate
- exit locked rate

---

7. FIFO / SWAP Logic

Code:
- [lib/logic.js](/home/koustubh/Downloads/website/scholar/lib/logic.js)

This is one of the most important behavior engines in the app.

Algorithm:

```
wrong pre-class questions
    !“
dedupe wrong questions
    !“
determine today's scheduled questions
    !“
prefer wrong questions not already in today's plan
    !“
swap them into today's session
    !“
push displaced questions to next class date
```

The most important invariant:
- swaps are anchored to `sessionDate`
- not to server clock

Risks

1. Date anchoring is correct in intent, but fragile in implementation.
   • Any accidental reintroduction of server-time logic breaks FIFO.

2. The app assumes question-level uniqueness on score rows.
   • If score rows are duplicated or stale, swaps can become inconsistent.

3. `applySwap()` writes multiple `pushQuestionToDate()` operations.
   • This is operationally expensive and can become slow under volume.

Metrics:
- swap trigger rate
- average number of displaced topics per swap
- swap write latency
- post-swap correctness rate (topics actually moved as expected)

---

8. Homework Logic

Code:
- [pages/api/student/homework.js](/home/koustubh/Downloads/website/scholar/pages/api/student/homework.js)

Homework is derived, not purely assigned.

Current stack:
- latest session questions
- weakness questions (`score > 2`)
- admin-uploaded homework (`hw_source = "admin_hw"`)

Algorithm:

```
load all question types with scores
    !“
find last session date <= today
    !“
 i f   e x i t   t i c k e t   f o r   t h a t   s e s s i o n   n o t   d o n e   !’   l o c k
    !“
union(session questions, weak questions, admin homework)
    !“
sort by priority
    !“
for each selected type:
    fetch cached MCQ only
         i f   n o   c a c h e   !’   s k i p
```

Important detail:
- homework is cache-dependent
- unlike assessment, it does not currently generate runtime MCQs when cache is missing

Risks

1. Homework can silently under-deliver if cache coverage is poor.
2. Union logic may overgrow with lots of weak topics.
3. The route does repeated per-question-type retrieval, which is latency-heavy.

Metrics:
- homework question count returned
- cache miss skip count
- homework lock rate
- per-request number of question types scanned

---

9. Practice Room / Progress Graph Logic

Code:
- [pages/api/student/progress-graph.js](/home/koustubh/Downloads/website/scholar/pages/api/student/progress-graph.js)
- [pages/api/student/progress-graph-attempt.js](/home/koustubh/Downloads/website/scholar/pages/api/student/progress-graph-attempt.js)
- [components/SubjectCylinder3D.js](/home/koustubh/Downloads/website/scholar/components/SubjectCylinder3D.js)

Algorithm:

```
load all question types with scores
    !“
for each question type:
    fetch stored questions by student context
    compute count + available daily question
    derive lock state
    !“
group into units / LOs / arcs on client
    !“
student attempts one daily question
    !“
persist attempt state
    !“
rerender 3D progress
```

Important hidden issue:
- this path depends heavily on `attempted_qhashes`-style state
- that field is not a stable mirrored Notion truth today

Risks

1. This is one of the most expensive APIs in the app.
- It does `getQuestionsForStudentContext(...)` for many question types.

2. It assumes a stable attempt-state store.
   • That assumption is weaker than the rest of the app.

3. UI cost is high.
   • 3D rendering + data hydration + re-render after each answer

Metrics:
- progress graph API latency
- number of question types scanned
- number of underlying page fetches
- 3D render frame rate on client
- interaction latency after answer submit

---

10. Admin Import Logic

Code:
- [pages/api/admin/import.js](/home/koustubh/Downloads/website/scholar/pages/api/admin/import.js)

Import is one of the densest logic paths in the product.

10.1 Import pipeline

```
PDF/DOCX upload
    !“
Claude extracts question types + questions + answers + taxonomy codes
    !“
readable names generated
    !“
existing subject question pages loaded
    !“
question type matched to existing page or new page created
    !“
page callout blocks written:
    Country / State / LO
    Q / A
    qhash
    !“
score row created for student + subject + date
    !“
MCQ cache pre-generation attempted
```

10.2 What import is really doing

It is performing all of these at once:
- OCR-like worksheet understanding
- question-type extraction
- taxonomy mapping
- naming

- deduplication
- question bank authoring
- score-row scheduling
- MCQ pre-generation

That is why it is both powerful and fragile.

Import-specific vulnerabilities

1. Taxonomy mapping is stochastic.
   - Claude chooses `standard_codes`.
   - This is not deterministic.

2. Question-type identity is fuzzy.
   - `original_type` in real data behaves like a content hash, mainly useful for re-imports.
   - if not a re-import, append/create logic falls back to fuzzy page-name matching

3. Import is computationally heavy.
   - PDF parse
   - Claude extraction
   - Notion writes
   - MCQ generation

4. It performs multi-step writes without a true transaction boundary.
   - Partial success is possible.

Metrics:
- import total duration
- Claude extraction duration
- number of question types extracted
- number of pages created vs appended
- post-import cache coverage
- import failure rate by stage

---

11. End-Class Logic

Code:
- [pages/api/admin/end-class.js](/home/koustubh/Downloads/website/scholar/pages/api/admin/end-class.js)

End class flow:

```
admin selects taught topics
    !“
latest session rows loaded
    !“
 t a u g h t   r o w s  !’  A p p r o v e d   f o r   E x i t
    !“
 u n t a u g h t   r o w s  !’  m o v e d   t o   n e x t   e s t i m a t e d   s e s s i o n
```

```
    !“
future rows may be pushed one slot forward
```

This is not just a status flip. It mutates schedule state.

Risks

1. Calendar-estimated future session dates are fuzzy.
2. Pushing future rows can cascade unexpectedly if schedule state is already messy.
3. This path is write-heavy and hard to reason about without explicit session entities.

Metrics:
- end-class duration
- number of rows moved
- number of rows approved
- mismatch rate between planned and taught topics

---

12. Calendar Matching

Code:
- [lib/tutor-calendar.js](/home/koustubh/Downloads/website/scholar/lib/tutor-calendar.js)

Calendar matching is heuristic:
- normalize title
- derive first few subject words
- fuzzy match student name

This is an operational helper, not a guaranteed identifier.

Vulnerability: fuzzy matching

This is the single most important fuzzy subsystem outside Claude.

Possible failure modes:
- false positive event match
- false negative event match
- duplicate match ambiguity
- similar student names
- similar subject names

This affects:
- dashboard timing
- Zoom links
- pre-class visibility assumptions
- end-class future-date estimates

Metrics:

- event match confidence (should be added)
- ambiguous event count
- unmatched class count

---

13. Deterministic vs Stochastic Subsystems

Deterministic-ish parts

- date filtering on score rows
- status changes
- FIFO swap mechanics
- weakness score increments
- question count from duration
- auth gating by email / session

Stochastic or fuzzy parts

- Claude taxonomy mapping
- Claude question-type extraction
- Claude MCQ generation
- import readable-name generation
- calendar event matching
- append/create matching in import
- context retrieval when page structure is inconsistent

Why this matters

The app mixes:
- hard state mutation
- fuzzy semantic interpretation

This is dangerous when fuzzy outputs are later treated as authoritative database state.

Recommendation:
- fuzzy systems should produce candidates + confidence
- deterministic systems should own final mutation when possible

---

14. Key Vulnerability Classes

14.1 Fuzzy matching vulnerabilities

Highest importance.

Examples:

- calendar event title matching
- import page similarity matching
- taxonomy mapping by Claude
- question-type naming equivalence
- LO/page-callout drift

Failure pattern:

- semantic ambiguity becomes persistent state

14.2 Schema drift vulnerabilities

Examples:

- page-body LO vs score-row standard_code
- cache format drift
- legacy pages without qhash
- app-native fields assumed to exist in Notion

Failure pattern:

- code silently interprets incomplete or inconsistent data

14.3 Auth / mode vulnerabilities

Examples:

- multiple auth modes: real, admin preview, local demo
- hardcoded admin email
- impersonation logic spread across routes

Failure pattern:

- access logic becomes hard to reason about

14.4 API rate and dependency vulnerabilities

External dependencies:

- Google OAuth
- Google Calendar API
- Anthropic API
- Notion API

Failure pattern:

- one slow or rate-limited dependency stalls the request path

14.5 Computational inefficiency

Main pattern:

- many endpoints perform repeated page fetches and repeated per-question-type operations

This is not always formally `O(n^2)`, but there are many `N x remote-call` patterns that behave badly in practice.

15. Where Compute / Latency Is Likely Worst

15.1 Progress graph

Pattern:
- load question types
- for each type, fetch question content

This is effectively:
- `O(n)` logical loop
- but with expensive remote calls inside
- so practical latency scales badly with more question types

15.2 Homework

Pattern:
- scan all subject question types
- derive union stack
- for each capped type, fetch context question pool

Again:
- logically moderate
- operationally expensive due to remote fetch cost

15.3 Import

Pattern:
- long-running multi-stage pipeline
- multiple sequential external operations

This is the most expensive end-to-end route in the app.

15.4 Assessment on cache miss

Pattern:
- fetch content
- call Claude synchronously

This creates high tail latency.

15.5 Admin review queue

Pattern:
- load score rows
- fetch related question pages
- parse sections and problems

This can get very slow for content-heavy subjects.

---

16. Complexity Notes

Examples of inefficient or risky patterns:

N+1 page fetches

Many flows do:

```
get all score rows
for each score row:
    fetch associated question page
```

This appears in various forms across:

- hydration
- progress graph
- review
- homework

Sequential external calls

Many flows await network calls in loops instead of batching aggressively.

Example risk:

- 20 question types
- 20 page reads
- 10 Claude calls
- 10 Notion writes

Even if formally linear, that can feel unusably slow.

Hidden cost of fallback logic

Fallbacks are good for robustness, but they add layers:

- exact context lookup
- full page fallback
- cache fallback
- runtime Claude fallback

This compounds latency.

---

17. Flow Diagrams

17.1 Student dashboard / assessment

```
Login
    !“
Dashboard API
    !“
Student + Subjects + Calendar
    !“
Choose mode
    % %    P r a c t i c e   R o o m
    % %    P r e - C l a s s
    % %    E x i t   T i c k e t
    % %    H o m e w o r k
```

17.2 Import

```
Admin uploads PDF/DOCX
    !“
Claude extracts question types
    !“
Taxonomy codes assigned
    !“
Question page matched or created
    !“
Q/A blocks + callouts written
    !“
Score row created
    !“
MCQ cache pre-generated
```

17.3 Practice Room

```
Load progress graph
    !“
Fetch question types + context questions
    !“
Build unit / LO / arc model
    !“
Student attempts one daily question
    !“
Persist attempt state
    !“
Rerender shapes
```

17.4 End class

```
Tutor selects taught topics
    !“
Mark taught rows "Approved for Exit"
    !“
Move untaught rows to next session
    !“
Potentially push later rows forward
```

```
    !“
Student exit ticket unlocks
```

---

18. Example Code Snippets That Matter

18.1 Auth gate

From [pages/api/auth/[...nextauth].js](/home/koustubh/Downloads/website/scholar/pages/api/auth/[...nextauth].js):

```
if ((user.email || "").toLowerCase() === "kbohuastt@gmail.com") return true
const student = await getStudentByEmail(user.email)
return !!student
```

This is simple and effective, but strongly tied to email matching.

18.2 FIFO anchoring

From [lib/logic.js](/home/koustubh/Downloads/website/scholar/lib/logic.js):

```
const sessionDate = todayQuestions[0]?.dateIntroduced || null
const targetDate = sessionDate
  ? getNextClassDateFrom(sessionDate, enrollmentDays || [])
  : (nextClassDate || getNextClassDateFrom(getTodayIST(), []))
```

This is an important correctness invariant.

18.3 Import writes LO to both page and score row

From [pages/api/admin/import.js](/home/koustubh/Downloads/website/scholar/pages/api/admin/import.js):

```
const loCode = Array.isArray(qt.standard_codes) ? qt.standard_codes.join(", ") :
(qt.standard_code || "")
const blocks = buildQuestionBlocks(qt, loCode, loName)
...
const scoreRow = await createScoreRow(studentId, subjectId, {
  id: questionPageId,
  title: pageTitle,
  standardCode: loCode,
  unit: qt.unit || "",
}, dateIntroduced, hwSource)
```

This matters because later backfills do not keep both in sync.

---

19. Metrics to Track for Concurrency and Latency

If multiple concurrent users should use this without insane latency, these metrics are essential.

Core API latency

Track per route:

- p50
- p95
- p99
- timeout rate

Highest-priority routes:

- `/api/student/dashboard`
- `/api/student/assessment`
- `/api/student/homework`
- `/api/student/progress-graph`
- `/api/admin/import`
- `/api/admin/review-queue`
- `/api/admin/end-class`

External dependency metrics

Track separately:

- Notion API latency
- Anthropic API latency
- Google Calendar API latency
- auth/session latency

Cache health

Track:

- MCQ cache hit rate
- runtime Claude generation rate
- cache backfill success rate
- image-only skipped count

Workload shape

Track:

- question types scanned per request
- question pages fetched per request
- Claude calls per request
- Notion writes per request
- imported question types per document

Correctness / drift metrics

Track:

- score rows with blank `standard_code`
- question pages with missing `qhash`

- pages with no callout sections
- rows whose `standard_code` does not appear in any page callout for the linked question type

User experience metrics

Track:
- assessment no-data rate
- homework no-data rate
- progress graph no-data rate
- calendar unmatched-subject rate
- auth/session refresh failure rate

---

20. Operational Recommendations Before Full Migration

Even before moving to a new DB, the following would reduce risk.

1. Instrument route latency and external call counts.
2. Add structured logs around:
   - page fetch counts
   - cache hits/misses
   - Claude fallback counts
3. Add confidence or diagnostics to fuzzy matchers.
4. Move more generation work off the student request path.
5. Reduce reliance on schema assumptions that are not actually true in Notion.

Most importantly:

```
Do not let fuzzy interpretation become silent permanent state
without logging, confidence, or auditability.
```

---

21. Final Judgment

The app is ambitious and genuinely complex.

Its complexity comes from:
- real product breadth
- layered workflows
- mixed sources of truth
- semantic AI in operational paths
- a data substrate doing more than it should

This is not a toy project and not a basic dashboard.

It is also exactly the kind of app that needs:

- sharper system documentation
- explicit metrics
- clearer ownership of state
- eventual migration to a more structured runtime store